W O R D L E

Build Your Own Wordle

A follow-along guide to build **your** Wordle: email + password login, a Daily Wordle that is the same valid word for every user, Custom Wordles of any length shared by link, and a dashboard with your solved count, win percentage and streaks. Backend in Python + FastAPI, with every guess judged on the server so the answer never reaches the browser.

Accounts

Register, login, bcrypt hashing, JWT sessions (Ch. 5)

Two game modes

Daily word for all + custom share links, any length (Ch. 6-8)

Dashboard

Solved 10/15, win %, streaks, guess distribution (Ch. 9)

> **How to use this guide:** Follow the chapters in order - each one ends with a checkpoint. Chapters 4-9 are the backend (database, auth, game engine). Chapters 10-11 are the frontend. Chapter 14 gives the exact build order: if you do those 10 steps in sequence, you will have a working game at every step and a finished product at the end.

Contents & What You Are Building

01 What You Are Building

02 Architecture At A Glance

03 Tech Stack (FastAPI)

04 Database Design

05 Auth - Email + Password

06 Daily Wordle Logic

07 Server-Side Guess Scoring

08 Custom Wordle & Share Links

09 Dashboard & Stats

10 Frontend - Pages & Flow

11 Frontend - Variable-Length Board

12 API Reference

13 Security & Edge Cases

14 Build Order & Deployment

The core loop

- Guess the hidden word within **6 tries**.
- After each guess, every tile is judged by the server.
- **Green** - right letter, right spot. **Yellow** - letter is in the word, wrong spot. **Grey** - letter is not in the word (duplicates handled by the two-pass rule, Ch. 7).

The product, in one page

SCREEN / MODE	WHAT THE USER DOES
Login / Register	Email + password. Nobody plays logged-out - every route checks login first.
Home	Chooses: Play Daily, Create Custom, or view Dashboard.
Daily Wordle	One **valid dictionary word** (5 letters), the **same for all users** on a given date.Plays once per day; result is saved to history.
Custom Wordle	Creator types **any word, any length** (no dictionary check), gets a share link like `/play/abc123`. Any logged-in user opening it can play. Creator sees who solved it.
Dashboard	Daily stats only: played, solved as **10/15**, win %, current streak, best streak, guess distribution.

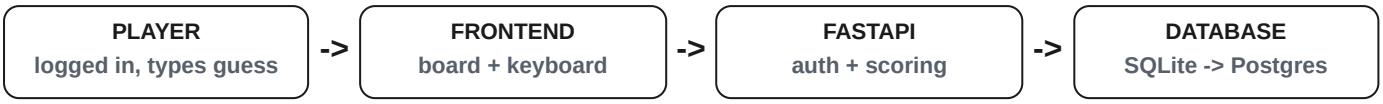
> **Scope lock for v1:** login, Daily, Custom + share links, Dashboard. Challenges, timed modes, leaderboards and multiplayer are deliberately left for later (Ch. 14 shows where they bolt on). Building v1 first is what makes those easy later.

What changed from a basic Wordle clone

BASIC CLONE	YOUR GAME (THIS GUIDE)
Anyone can play, no account	Login required; every game and stat belongs to a user
Answer sits in browser JavaScript	Answer lives only on the server; browser receives tile results, never the word
Fixed 5-letter board	Board builds itself for any word length (Daily = 5, Custom = 3-10)
Stats in localStorage, lost on new phone	Stats computed from the database - same on every device

Architecture & Tech Stack

The golden rule: **the frontend never knows the answer until the game ends.** If the answer were in the page source, anyone could cheat with View Source. FastAPI owns the secrets and the truth; the frontend owns the experience.



Frontend responsibilities

- Login / register forms; store the JWT and send it on every request
- Render the board for any word length; capture physical + on-screen keyboard
- Send each guess to the server; render the tile results it returns
- Animate flips, shake on invalid guess, win bounce
- Dashboard charts from the stats API

Backend responsibilities (FastAPI)

- Register / login; hash passwords; issue and verify JWTs
- Pick one valid daily word per date - same for everyone
- Hold every answer; score guesses server-side (Ch. 7)
- Create custom games; resolve share links
- Save results; compute dashboard stats
- Rate-limit guess endpoints against brute force

The stack

LAYER	CHOICE	WHY
Backend	**Python + FastAPI** + Uvicorn	You know Python; FastAPI gives typed endpoints and free API docs at `/docs`
Database	SQLite to start, Postgres later (SQLAlchemy)	Zero-setup locally; one connection-string change to go live
Auth	bcrypt (passlib) + JWT (python-jose)	Industry-standard hashing; stateless tokens, no session table needed
Frontend	Vanilla HTML/CSS/JS (or React later)	The game logic is one board component; no framework needed for v1
Word list	Text file of valid 5-letter words, server-side only	Daily answers drawn from it; guesses validated against it

Project structure

```
my-wordle/
  backend/
    main.py          # FastAPI app - all endpoints (Ch. 12)
    models.py        # SQLAlchemy tables (Ch. 4)
    auth.py          # hashing + JWT helpers (Ch. 5)
    game.py          # daily picker + two-pass scorer (Ch. 6-7)
    words.txt        # valid words - SERVER ONLY, never served
    requirements.txt # fastapi uvicorn sqlalchemy passlib[bcrypt]
                     # python-jose[cryptography] python-multipart
  frontend/
    index.html       # login + home
    game.html        # shared board page (daily + custom)
    dashboard.html   # stats
    app.js           # api helper, board, keyboard
    styles.css
  README.md
```

> **Run locally:** `pip install -r requirements.txt` then `uvicorn main:app --reload`. Open `http://127.0.0.1:8000/docs` - FastAPI's interactive API page is your best debugging friend throughout this build.

Database Design

Five tables hold the whole product. All game writes happen on the server, after it has judged the guess - the client never writes a result directly.

TABLE	COLUMNS	PURPOSE
`users`	id, email (unique), password_hash, display_name, created_at	Accounts. Only the hash is stored, never the password.
`daily_words`	word_date (PK), word	One valid word per date, filled by the picker (Ch. 6). Same row served to everyone.
`daily_results`	id, user_id, word_date, won, guesses_used, finished_at. Unique(user_id, word_date)	One finished Daily per user per day - dashboard raw material.
`custom_games`	id, creator_id, answer, length, share_code (unique), created_at	Creator's word, server-only. share_code is the public link id.
`custom_attempts`	id, game_id, player_id, won, guesses_used, finished_at. Unique(game_id, player_id)	Who played a custom game and how it ended.

Why daily_words is a table, not a formula

A table lets you curate (no repeat of yesterday's word) and audit what was served. The picker fills rows in advance (Ch. 6).

Why attempts store only the ending

Guesses are judged live and discarded; persist only won + guesses_used. All the dashboard needs - and the tables stay tiny.

Database Design - The Models In Code

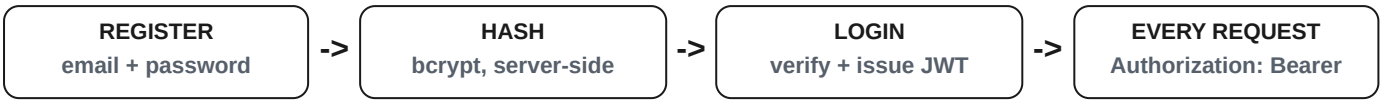
SQLAlchemy models (models.py)

```
from sqlalchemy import Column, Integer, String, Boolean, Date, DateTime, ForeignKey, UniqueConstraint
from sqlalchemy.orm import declarative_base
from datetime import datetime
Base = declarative_base()
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    email = Column(String, unique=True, index=True, nullable=False)
    password_hash = Column(String, nullable=False)
    display_name = Column(String, default="")
    created_at = Column(DateTime, default=datetime.utcnow)
class DailyWord(Base):
    __tablename__ = "daily_words"
    word_date = Column(Date, primary_key=True)
    word = Column(String(5), nullable=False)  # valid dictionary word
class DailyResult(Base):
    __tablename__ = "daily_results"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    word_date = Column(Date); won = Column(Boolean)
    guesses_used = Column(Integer)  # 1-6
    finished_at = Column(DateTime, default=datetime.utcnow)
    __table_args__ = (UniqueConstraint("user_id", "word_date"),)
class CustomGame(Base):
    __tablename__ = "custom_games"
    id = Column(Integer, primary_key=True)
    creator_id = Column(Integer, ForeignKey("users.id"))
    answer = Column(String(10), nullable=False)  # ANY word, length 3-10
    length = Column(Integer, nullable=False)
    share_code = Column(String(12), unique=True, index=True)
    created_at = Column(DateTime, default=datetime.utcnow)
class CustomAttempt(Base):
    __tablename__ = "custom_attempts"
    id = Column(Integer, primary_key=True)
    game_id = Column(Integer, ForeignKey("custom_games.id"))
    player_id = Column(Integer, ForeignKey("users.id"))
    won = Column(Boolean); guesses_used = Column(Integer)
    finished_at = Column(DateTime, default=datetime.utcnow)
    __table_args__ = (UniqueConstraint("game_id", "player_id"),)
```

> **Checkpoint:** Run `Base.metadata.create_all(engine)` and open the SQLite file - all five tables should exist. Every later chapter assumes these exact tables.

Auth - Email + Password

Every screen except Login/Register requires login. Register stores only a **bcrypt hash**; login returns a **JWT** the frontend sends with every request; FastAPI verifies it before any game logic runs.



Password hashing + JWT helpers (auth.py)

```
from passlib.context import CryptContext
from jose import jwt, JWTError
from datetime import datetime, timedelta
import os

SECRET = os.environ["JWT_SECRET"]            # long random string, never in git
ALGO, EXPIRE_DAYS = "HS256", 7
pwd_ctx = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(p): return pwd_ctx.hash(p)
def verify_password(p, h): return pwd_ctx.verify(p, h)

def make_token(user_id):
    payload = {"sub": str(user_id),
               "exp": datetime.utcnow() + timedelta(days=EXPIRE_DAYS)}
    return jwt.encode(payload, SECRET, algorithm=ALGO)

def read_token(token):                        # returns user_id or None
    try: return int(jwt.decode(token, SECRET, algorithms=[ALGO])["sub"])
    except (JWTError, KeyError, ValueError): return None
```

Auth - Endpoints & Rules

The three auth endpoints

```
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer
bearer = HTTPBearer()
def current_user(cred=Depends(bearer), db=Depends(get_db)):
    uid = read_token(cred.credentials)
    user = db.get(User, uid) if uid else None
    if not user: raise HTTPException(401, "Login required")
    return user  # Depends(current_user) protects any endpoint
@app.post("/api/auth/register")
def register(data: RegisterIn, db=Depends(get_db)):
    if db.query(User).filter_by(email=data.email.lower()).first():
        raise HTTPException(400, "Email already registered")
    user = User(email=data.email.lower(), display_name=data.display_name,
                password_hash=hash_password(data.password))
    db.add(user); db.commit()
    return {"token": make_token(user.id), "user": {"id": user.id, "email": user.email}}
@app.post("/api/auth/login")
def login(data: LoginIn, db=Depends(get_db)):
    user = db.query(User).filter_by(email=data.email.lower()).first()
    if not user or not verify_password(data.password, user.password_hash):
        raise HTTPException(401, "Wrong email or password")  # same either way
    return {"token": make_token(user.id), "user": {"id": user.id, "email": user.email}}
@app.get("/api/auth/me")
def me(user=Depends(current_user)):
    return {"id": user.id, "email": user.email, "display_name": user.display_name}
```

Validation rules (use Pydantic)

Email must parse (`EmailStr`); password minimum 8 characters. Lowercase emails at register and login so `A@x.com = a@x.com` .

Frontend token handling

Save the token in `localStorage["token"]` . Every fetch adds the Bearer header. On any 401, clear it and redirect to login. Never store the password.

> **Not optional:** HTTPS in production, `JWT_SECRET` from an env var, identical "wrong email or password" errors, and rate-limiting `/api/auth/login` (10/min/IP).

Daily Wordle - One Valid Word For Everyone

The Daily is the shared ritual: on a given date, **every user plays the same word**, always a **real, valid word** from the curated list. Clients ask "what is today's game?", never "what is today's word?".

The word list

Keep one server-side file, `words.txt` : valid 5-letter words, lowercase, one per line (about 2,000-2,300 common answers is the classic size; a larger allowed-guess list is optional in v1 - simplest correct v1 rule: a Daily guess must be in the same list). This file is **never** sent to the browser. Curate it once: common, guessable words; no plurals ending in S; no offensive words as answers.

Daily word selection (game.py)

```
from datetime import date

WORDS = [w.strip() for w in open("words.txt") if len(w.strip()) == 5]
WORD_SET = set(WORDS)
EPOCH = date(2024, 1, 1)   # your launch date - freeze it forever

def pick_daily_word(d: date) -> str:
    # Deterministic: same date -> same word, on any server, forever.
    days = (d - EPOCH).days
    return WORDS[days % len(WORDS)]

def get_or_create_daily(db, d: date) -> str:
    row = db.get(DailyWord, d)
    if not row:
        row = DailyWord(word_date=d, word=pick_daily_word(d))
        db.add(row); db.commit()
    return row.word

def daily_status(db, user, today: date):
    get_or_create_daily(db, today)
    played = db.query(DailyResult).filter_by(
        user_id=user.id, word_date=today).first()
    return {"date": str(today), "length": 5, "max_guesses": 6,
            "already_played": played is not None,
            "won": played.won if played else None}
```

Why this picker is safe

The modulo walk visits words in list order; shuffle WORDS once with a fixed seed before launch so the sequence is not alphabetical, then freeze the file. Adding words later shifts future picks - append new words at the end only, or accept a one-time reshuffle.

Timezone rule

"Today" is the **server's date** in v1 - one global puzzle that flips at server midnight; simple and identical for all. (Per-user timezone is a later refinement; it complicates the shared-ritual guarantee.)

Playing the Daily

1. Frontend calls `GET /api/daily/today` - date, length 5, already-played state. **No word in the response.**
2. Each guess goes to `POST /api/daily/guess` (Ch. 7): valid 5 letters, in WORD_SET, not already finished.
3. On win or 6th guess, the server writes the `daily_results` row (unique per user+date - no double-count) and only then reveals the word.
4. Dashboard numbers move immediately - they read the same table (Ch. 9).

> **Checkpoint:** Log in as two different users on the same day - both must face the same hidden word (test by finishing as user 1 and comparing the revealed word with user 2's). Then try to find the word in the browser's Network tab before finishing: you should see only tile results, never the answer.

Server-Side Guess Scoring - The Engine

Every guess - Daily or Custom, 5 letters or 9 - is judged by **one function on the server**. The client sends a guess; the server replies with per-tile results and nothing else: anti-cheat plus one board for any word length.

The two-pass algorithm, any length (game.py)

```
def score_guess(guess: str, answer: str) -> list:
    # Both uppercase, same length. Returns per-tile:
    # "correct" | "present" | "absent"
    n = len(answer)
    result = ["absent"] * n
    remaining = {}
    # PASS 1: greens first; bank unmatched answer letters
    for i in range(n):
        if guess[i] == answer[i]:
            result[i] = "correct"
        else:
            remaining[answer[i]] = remaining.get(answer[i], 0) + 1
    # PASS 2: yellows only while banked letters remain
    for i in range(n):
        if result[i] == "correct":
            continue
        if remaining.get(guess[i], 0) > 0:
            result[i] = "present"
            remaining[guess[i]] -= 1
    return result
# score_guess("BABES", "ABBEY")
# -> ["present", "present", "correct", "correct", "absent"]
```

Why two passes are non-negotiable

A naive "yellow if the letter exists" over-reports duplicates. Answer ABBEY, guess BABES: there are only two Bs; pass 1 consumes one as green, the bank allows exactly one yellow. This function is length-agnostic - the same code scores a 3-letter and a 10-letter custom game.

What the server returns

`{ result: [...], status: "playing"|"won"|"lost", guesses_used, answer }` - with `answer` included **only** when status is won/lost. Plus `404/422` errors: unknown game, wrong length, Daily guess not in the valid list.

The shared guess endpoint pattern

```
def judge(db, user, answer, guess, max_guesses, state):
    # state = the player's running game (kept server-side, see below)
    g = guess.upper().strip()
    if len(g) != len(answer) or not g.isalpha():
        raise HTTPException(422, "Guess must be %d letters" % len(answer))
    state["guesses"].append(g)
    result = score_guess(g, answer)
    won = all(s == "correct" for s in result)
    over = won or len(state["guesses"]) >= max_guesses
    return result, won, over, len(state["guesses"])
```

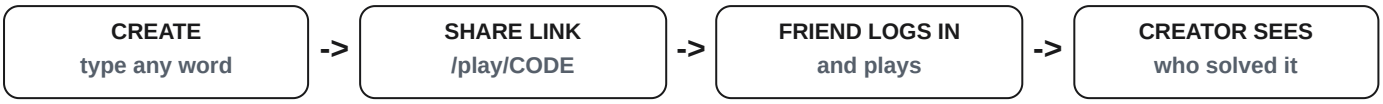
> **Where does in-progress state live?** For v1, keep it simple and honest: the client holds the list of guesses it has made, and sends `guesses_so_far` count with each guess; the server validates the count and, at game over, the client posts the finished guesses for re-verification before the result row is written. The stronger v2 (server-side game sessions table) is noted in Ch. 13 - v1's rule already prevents the one cheat that matters: the answer is never exposed, and final results are re-scored server-side before saving.

Daily-specific checks before judging

- Guess must be in `WORD_SET`, else `422 "Not in word list"` - and the guess does **not** consume a turn.
- User must not already have a `daily_results` row for today, else `409 "Already played today"`.
- Custom games skip the dictionary check entirely (Ch. 8) but keep length + letters-only checks.

Custom Wordle - Any Word, Shareable By Link

Custom Wordle is the social half: a logged-in user invents a word - **any length, any letters, no dictionary check** - and gets a link. Visitors must be **logged in** to play, on the same board and engine.



Creating a custom game

```
import secrets, string
def new_share_code(db):
    alphabet = string.ascii_lowercase + string.digits
    while True:
        code = "".join(secrets.choice(alphabet) for _ in range(8))
        if not db.query(CustomGame).filter_by(share_code=code).first():
            return code

@app.post("/api/custom")
def create_custom(data: CustomIn, user=Depends(current_user), db=Depends(get_db)):
    word = data.word.upper().strip()
    if not (3 <= len(word) <= 10) or not word.isalpha():
        raise HTTPException(422, "Word must be 3-10 letters (A-Z only)")
    game = CustomGame(creator_id=user.id, answer=word,
                      length=len(word), share_code=new_share_code(db))
    db.add(game); db.commit()
    return {"share_code": game.share_code, "url": "/play/" + game.share_code,
            "length": game.length}
```

Opening a shared link

```
@app.get("/api/custom/{code}")
def open_custom(code: str, user=Depends(current_user), db=Depends(get_db)):
    game = db.query(CustomGame).filter_by(share_code=code).first()
    if not game: raise HTTPException(404, "Game not found")
    mine = db.query(CustomAttempt).filter_by(
        game_id=game.id, player_id=user.id).first()
    return {"length": game.length, "max_guesses": 6,
            "creator": game.creator_id == user.id,
            "already_played": mine is not None,
            "won": mine.won if mine else None}  # no answer here

@app.post("/api/custom/{code}/guess")
def guess_custom(code: str, data: GuessIn, user=Depends(current_user), db=Depends(get_db)):
    game = db.query(CustomGame).filter_by(share_code=code).first()
    if not game: raise HTTPException(404, "Game not found")
    # NO dictionary check - any word is legal.
    result, won, over, used = judge(db, user, game.answer,
        data.guess, 6, {"guesses": list(data.guesses)})
    if over:  # write the attempt once (unique game+player)
        save_attempt(db, game.id, user.id, won, used)
    return {"result": result, "status": "won" if won else "lost" if over else "playing",
            "guesses_used": used, **({"answer": game.answer} if over else {})}
```

The rules, stated plainly

Length 3-10, A-Z only (uppercased). No dictionary check - "PIKACHU" is a legal custom word. Max guesses stays 6 at every length in v1.

Access rules

Creating and opening links both require login (logged-out visitors detour via login, then land back on the game). `GET /api/custom/mine` lists the creator's games with solver counts.

> **Checkpoint:** Create a 7-letter custom word and open its link in a private window - login first, then a 7-wide board that accepts a made-up guess like "ZQXJKVB".

Dashboard - Scores, Streaks & Distribution

The dashboard answers one question: **how am I doing at the Daily?** Everything is computed from `daily_results` - no separate stats table, so numbers can never disagree with history. Custom games get their own lists and never pollute Daily stats.

10/15

Solved / Played
15 dailies attempted, 10 won

67%

Win percentage
wins / played x 100

4 - 9

Current / best streak
consecutive daily wins

Computing the stats (one endpoint)

```
@app.get("/api/stats/me")
def my_stats(user=Depends(current_user), db=Depends(get_db)):
    rows = (db.query(DailyResult).filter_by(user_id=user.id)
              .order_by(DailyResult.word_date).all())
    played = len(rows)
    wins = sum(1 for r in rows if r.won)
    dist = [0]*6
    for r in rows:
        if r.won: dist[r.guesses_used - 1] += 1
    # streaks: walk dates backwards from the most recent play
    streak = best = run = 0
    prev = None
    for r in rows:  # ascending by date
        if r.won and (prev is None or (r.word_date - prev).days == 1):
            run += 1
        elif r.won:
            run = 1
        else:
            run = 0
        prev = r.word_date
        best = max(best, run)
    streak = run if rows and rows[-1].won else 0
    return {"played": played, "wins": wins,
            "solved_label": "%d/%d" % (wins, played),
            "win_pct": round(100 * wins / played) if played else 0,
            "current_streak": streak, "max_streak": best,
            "guess_distribution": dist,
            "history": [{"date": str(r.word_date), "won": r.won,
                         "guesses": r.guesses_used} for r in reversed(rows)]}
```

Streak rule (match player expectations)

A streak counts consecutive **calendar days** each ending in a win. Missing a day or losing resets the current streak to 0; the best streak is kept forever. The walk above enforces this with the `(date - prev).days == 1` check.

Guess distribution

Six bars - games won in 1, 2, 3, 4, 5, 6 guesses. Render as horizontal bars scaled to the largest bucket; highlight today's bar after a win. Losses appear in history but in no bar.

What the dashboard page shows

- **Top cards:** Solved (10/15 label), Win %, Current streak, Best streak, Total played.
- **Guess distribution** bar chart (6 rows).
- **History table:** date, Won/Lost, guesses used - newest first. Unplayed dates do not appear.
- **Custom summary:** games I created (with solver counts) and custom games I played, kept separate from Daily stats.

> **Checkpoint:** Finish today's Daily, open the dashboard - played increments by exactly 1, and refreshing or re-opening the game cannot increment it again (the unique user+date constraint is doing that work).

Frontend - Pages, Flow & The API Helper

The frontend is five screens sharing one board component and one API helper. Keep it framework-free for v1: plain pages, one `app.js`, one stylesheet. Every page except Login/Register first checks for a token - no token, straight to login.

PAGE	ROUTE	WHAT IT DOES
Login / Register	`/login`, `/register`	Posts to `/api/auth/*`, saves the JWT, redirects home. Register asks email, password, display name.
Home	`/` (logged in)	Three big choices: Play Daily (shows "Played today" state), Create Custom, Dashboard. Plus "My custom games".
Daily game	`/daily`	5-wide board; loads `/api/daily/today`; finished state shown if already played.
Create custom	`/create`	One input: the word (3-10 letters). Returns the share link with a Copy button.
Play custom	`/play/{code}`	Same board, width = game length. This is the share-link landing page.
Dashboard	`/dashboard`	Stats cards, distribution bars, history, my custom games (Ch. 9).

The API helper - write once, use everywhere (app.js)

```
const API = "";  // same origin in dev via proxy, or your backend URL
function token() { return localStorage.getItem("token"); }
async function api(path, options = {}) {
  const res = await fetch(API + path, {
    ...options,
    headers: { "Content-Type": "application/json",
               "Authorization": "Bearer " + token(),
               ...(options.headers || {}) } });
  if (res.status === 401) {          // expired / logged out
    localStorage.removeItem("token");
    location.href = "/login?next=" + encodeURIComponent(location.pathname);
    return;
  }
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(data.detail || "Request failed");
  return data;
}
// usage: const me = await api("/api/auth/me");
//        await api("/api/daily/guess", {method:"POST", body: JSON.stringify({guess, guesses: past})});
```

The "next" redirect

Share links must survive login: a logged-out visitor opening `/play/abc123` is sent to `/login?next=/play/abc123`, and after login returns to the game. Without this, shared links feel broken.

Route guard

Two lines at the top of every protected page: `if (!token()) location.href = "/login?next=" + location.pathname;` The server still verifies the JWT on every API call - the guard is UX, the 401 check is security.

Login form sketch

```
form.addEventListener("submit", async (e) => {
  e.preventDefault();
  try {
    const data = await api("/api/auth/login", { method: "POST",
      body: JSON.stringify({ email: emailEl.value.trim(),
                             password: passwordEl.value }) });
    localStorage.setItem("token", data.token);
    location.href = new URLSearchParams(location.search).get("next") || "/";
  } catch (err) { showToast(err.message); }   // "Wrong email or password"
});
```

Frontend - The Variable-Length Board

One board powers Daily (5) and Custom (3-10): built from a server-returned `wordLength`.

Building the board for any length

```
function buildBoard(wordLength, maxGuesses = 6) {
  boardEl.innerHTML = "";
  boardEl.style.gridTemplateRows = "repeat(" + maxGuesses + ", 1fr)";
  for (let r = 0; r < maxGuesses; r++) {
    const row = document.createElement("div");
    row.className = "row";
    row.style.gridTemplateColumns = "repeat(" + wordLength + ", 1fr)";
    for (let c = 0; c < wordLength; c++) {
      const t = document.createElement("div");
      t.className = "tile"; row.append(t);
    }
    boardEl.append(row);
  }
  boardEl.style.width = Math.min(62 * wordLength + 30, 92) + "vw";
}
```

State + submit flow

```
const game = { length: 5, max: 6, row: 0, current: "",
               over: false, past: [], keyStates: {} };

async function submitGuess() {
  if (game.over || game.current.length !== game.length) return;
  try {
    const data = await api(game.endpoint + "/guess", { method: "POST",
      body: JSON.stringify({ guess: game.current, guesses: game.past }) });
    paintRow(game.row, data.result);       // stagger flips 250ms apart
    updateKeyboard(game.current, data.result);
    game.past.push(game.current); game.current = "";
    if (data.status !== "playing") {
      game.over = true;
      showEnd(data.status === "won", data.answer, data.guesses_used);
    }
    game.row++;
  } catch (err) { shakeRow(game.row); showToast(err.message); }
  // 422 "Not in word list" (Daily) or "Must be N letters":
  // the turn is NOT consumed - row and current stay untouched.
}
```

Keyboard priority

A letter keeps its best-ever state: correct > present > absent - never downgrade a green key. The keyboard is just letters, Enter, Backspace at any length.

CSS that adapts

`.row{display:grid;gap:5px}`, columns set inline per game. Tile font scales via a CSS variable. Flip/shake/pop/bounce keyframes are length-independent.

Colour tokens (match the classic feel)

```
:root{ --green:#538d4e; --yellow:#c9b458; --grey:#787c7e;
  --border:#d3d6da; --bg:#ffffff; --text:#1a1a1b; }
.tile.correct{background:var(--green);color:#fff;border-color:var(--green)}
.tile.present{background:var(--yellow);color:#fff;border-color:var(--yellow)}
.tile.absent {background:var(--grey);color:#fff;border-color:var(--grey)}
/* Colour-blind option: green -> #f5793a, yellow -> #85c0f9 */
```

> **Checkpoint:** Load a 5-letter Daily and a 9-letter Custom back to back - same code, both render; a Daily "Not in word list" rejection leaves the row untouched, a Custom made-up guess is accepted.

API Reference - The Complete Contract

Ten endpoints run the entire product. Everything under `/api/daily`, `/api/custom` and `/api/stats` requires `Authorization: Bearer <JWT>`. Test any of them from FastAPI's `/docs` page while building - click Authorize, paste a login token, and exercise endpoints without writing frontend code first.

ENDPOINT	AUTH	REQUEST -> RESPONSE
`POST /api/auth/register`	-	`{email, password, display_name}` -> `{token, user}`. 400 if email taken.
`POST /api/auth/login`	-	`{email, password}` -> `{token, user}`. 401 generic message.
`GET /api/auth/me`	JWT	-> `{id, email, display_name}`
`GET /api/daily/today`	JWT	-> `{date, length:5, max_guesses:6, already_played, won}`. Never the word.
`POST /api/daily/guess`	JWT	`{guess, guesses[]}` -> `{result[], status, guesses_used, answer?}`. 422 not in word list; 409 already finished.
`POST /api/custom`	JWT	`{word}` (3-10 letters) -> `{share_code, url, length}`
`GET /api/custom/mine`	JWT	-> my games: `[{share_code, length, created_at, players, solvers}]` (word shown only to creator)
`GET /api/custom/{code}`	JWT	-> `{length, max_guesses, creator, already_played, won}`. 404 unknown code.
`POST /api/custom/{code}/guess`	JWT	`{guess, guesses[]}` -> `{result[], status, guesses_used, answer?}`. No dictionary check.
`GET /api/stats/me`	JWT	-> `{played, wins, solved_label "10/15", win_pct, current_streak, max_streak, guess_distribution[6], history[]}`

Request/response schemas (Pydantic, main.py)

```
from pydantic import BaseModel, EmailStr

class RegisterIn(BaseModel):
    email: EmailStr
    password: str            # min_length enforced in endpoint: >= 8
    display_name: str = ""

class LoginIn(BaseModel):
    email: EmailStr
    password: str

class CustomIn(BaseModel):
    word: str                # validated: 3-10 chars, A-Z after upper()

class GuessIn(BaseModel):
    guess: str
    guesses: list = []       # prior guesses this game (for re-scoring)
```

Finishing a game - verify, then save

When a guess ends a game, the server re-scores the full guess list against the true answer before writing the result row. Match - save; mismatch - reject with 400. This closes the "claim a win I did not earn" hole for one extra scoring loop.

> **Error vocabulary (keep it consistent):** 400 bad input / duplicate, 401 login required or wrong credentials, 404 unknown game code, 409 already played, 422 validation failed (wrong length, not in word list), 429 rate limited. The frontend turns `detail` straight into a toast message.

Security, Validation & Edge Cases

Anti-cheat checklist

- Answers (`words.txt`, custom answers, today's daily word) exist only in server memory/database - never in HTML, JS, or any pre-game-over response
- Scoring happens only on the server; the client renders what it is given
- Finished results are re-scored server-side before saving (Ch. 12)
- Rate-limit guesses (about 60/minute/user) and logins (10/minute/IP) - use `slowapi` or middleware
- One Daily result per user per date, one Custom attempt per user per game - enforced by database unique constraints, not just code

Validation rules

- Email format + lowercase normalisation; password at least 8 characters
- Daily guess: exactly 5 letters A-Z and in the valid word set
- Custom word: 3-10 letters A-Z only (after upper-casing); no spaces or digits in v1
- Guess endpoints reject play after a game is finished, and Daily rejects a second play the same day (409)
- Share codes are random (not sequential ids) so links cannot be guessed or enumerated

Edge cases that will find you if you do not find them first

CASE	CORRECT BEHAVIOUR
Duplicate letters (answer ABBEY, guess BABES)	Two-pass scoring: present, present, correct, correct, absent (Ch. 7). Test this first.
Player refreshes mid-game	v1: board resets (acceptable, note it). Better: keep today's guesses in localStorage keyed by date / share code and replay the returned results.
Two tabs play the same Daily simultaneously	Unique constraint saves one result; the second finish gets 409 and shows the recorded result.
Creator opens their own share link	Allowed: show length and their solver list; playing their own game is permitted but flagged `creator: true` in stats views.
Missed a day / lost yesterday	Current streak reads 0 on the dashboard; best streak preserved (date-gap check in Ch. 9).
Token expires mid-session (7 days)	Any API call returns 401; the frontend helper clears the token and redirects to login with `?next=` back to the same page.
Custom word with repeated letters, length 9	The same two-pass scorer handles it - no special case. Board renders 9 columns.

Test cases to write before you ship

TEST	EXPECT
`score_guess("BABES","ABBEY")`	present, present, correct, correct, absent
`score_guess("ZQXJKVB","PIKACHU")` (custom)	Length-7 result, no dictionary rejection
Register twice with same email (different case)	400 "Email already registered"
Daily guess without token / with expired token	401
Finish Daily, submit again	409; dashboard played count unchanged
Open `/play/unknowncode` logged in	404 "Game not found"

Build Order, Deployment & What Comes Next

Build in this order - a working game at every step

1. **Skeleton:** FastAPI app + SQLite + `create_all`. Checkpoint: `/docs` loads. (30 min)
2. **Auth:** register, login, `/me`, the `current_user` dependency. Checkpoint: login in `/docs` with Authorize and call `/me`.
3. **Scorer:** `score_guess` + the Ch. 13 tests (ABBEY/BABES first).
4. **Daily:** word list, picker, `/daily/today`, `/daily/guess`, result saving. Checkpoint: two users, same hidden word.
5. **Frontend board:** variable-length board + keyboard wired to the Daily endpoints. Checkpoint: a full game in the browser, answer invisible in Network.
6. **Login pages:** register/login UI, token storage, route guards, `?next=` redirect.
7. **Custom:** create endpoint, share code, `/play/{code}` page reusing the board. Checkpoint: 7-letter game via link in a private window.
8. **Dashboard:** `/stats/me` + cards, distribution, history, my-custom-games. Checkpoint: finish a Daily, numbers move by exactly one.
9. **Hardening:** rate limits, finish re-verification, error toasts, mobile pass (360px wide, no keyboard zoom).
10. **Deploy** (below), then play it on your phone - the real final test.

Deployment

PIECE	SIMPLEST HOST	NOTES
FastAPI backend	Render / Railway / a small VPS	Start: `uvicorn main:app --host 0.0.0.0 --port $PORT`. Set `JWT_SECRET` as an env var.
Database	SQLite on the same host to start; Postgres (Neon / Supabase free tier) when you grow	SQLAlchemy means switching is a connection-string change. Back up the file/DB - it holds all user history.
Frontend	Serve from FastAPI itself (`StaticFiles`) or Netlify/Vercel	Same-origin serving avoids CORS entirely for v1 - simplest deploy wins.
HTTPS	Automatic on all hosts above	Required: passwords and JWTs must never travel over plain HTTP.

Where future modes bolt on (no redesign needed)

- **Timed / challenge modes:** a `mode` field on results + a timer in the board component; the scorer does not change.
- **Daily leaderboard:** aggregate `daily_results` by date - wins, then fewest guesses. One query.
- **Head-to-head custom battles:** two attempts on one custom game, compare `guesses_used` - the attempts table already stores both.
- **Server-side game sessions:** replace the client-held guess list with a `games_in_progress` table when you want refresh-proof resume and stricter anti-cheat.
- **Per-user timezone Daily:** store timezone at registration; the picker keys off the user's local date.
- **Word of any script / emoji mode:** length-agnostic scorer already copes; only input validation widens.

Glossary

TERM	MEANING
Valid word (Daily)	A word from the curated server-side list - daily answers and accepted daily guesses
Two-pass scoring	Greens first, then yellows against remaining letters - the only correct duplicate handling, any length
Share code	Random 8-character id in a custom game's link (`/play/{code}`)
Solved label	Dashboard headline "10/15" = daily wins / daily games played

> **You have the whole product on paper.** Build the steps in order, check each checkpoint, and your Wordle is live. When v1 feels solid, pick your first challenge mode from the list above.